

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Final Project Report

Smart Lost & Found

Team: **Murad/Yasin**
Repo: **[Repo Link](#)**
Members: **[Murad Taghizada profile](#)**
[Yasin Veliyev profile](#)

Date: **23.05.2026**
Tag: v1.0-final

Executive Summary

Required. Exactly one paragraph (5–7 sentences). What you built, the headline concurrency speedup you measured, your headline robustness story, and the single most surprising thing the team learned. *This is the first thing the grader reads. If you cannot write it clearly, you do not yet understand your own project.*

Your paragraph should answer: What problem did the system solve? What is the one number that proves the concurrency design works? What is the one failure mode you handled well? What would you change if you started over?

Smart Lost & Found is a backend system designed to connect lost and found items by leveraging AI-generated item descriptions and embedding-based similarity matching. Users can submit lost or found item reports through either a FastAPI REST API or a command-line interface. Uploaded images are processed by a vision-language model, which generates descriptive metadata and embedding vectors that can be efficiently searched and compared. Our work concentrated on the software engineering infrastructure surrounding the provided AI components, including input validation, retry mechanisms, asynchronous processing, PostgreSQL integration, caching, logging, containerization with Docker, and support for offline testing. To improve performance, we implemented controlled asynchronous execution using `asyncio.gather` together with `asyncio.Semaphore`, achieving roughly a fivefold improvement over a purely sequential approach for I/O-intensive tasks. A key reliability feature was graceful degradation: if PostgreSQL was unavailable during development, the system could automatically switch to JSON-based storage rather than failing outright. One of the biggest insights from the project was realizing that much of the complexity came not from the AI model itself, but from handling external service failures, rate limiting, retry strategies, data persistence, and reproducible offline execution. Looking back, we would invest more effort from the beginning in observability, monitoring, and benchmarking capabilities.

Contents

Executive Summary	1
1 Project Overview	2
1.1 Problem framing & scope	2
1.2 Provider choice and the AI module contract	3

2 Architecture	3
2.1 Module map.....	3
2.2 OOP design & key abstractions.....	4
2.3 Data flow across module boundaries	4
3 Concurrency & Performance	5
3.1 Why this concurrency model.....	5
3.2 Sequential-vs-concurrent benchmark.....	5
3.3 Rate limit & politeness.....	6
4 Robustness & Error Handling	6
4.1 Wrapping the AI module.....	6
4.2 Failure-mode analysis (one concrete incident).....	7
4.3 Input validation	7
5 Storage & Caching	7
5.1 Schema.....	7
5.2 Caching policy.....	8
6 Testing	8
6.1 Strategy & coverage.....	8
6.2 Notable tests.....	8
7 Deployment & Reproducibility	9
7.1 Docker image.....	9
7.2 Configuration	9
8 Limitations & Future Work	9
9 Tools & Acknowledgements	10
References	10
A Appendix — Reproducing the benchmark	10

1. Project Overview

1.1 Problem framing & scope

Required. 2–3 paragraphs. State the topic problem in your own words. List the inputs, outputs, and any user-facing entry points (CLI commands, HTTP endpoints). State which provider stack you used (LLM, embedding, USDA, web search).

Beyond the obvious: which parts of the TOPIC.md spec did your team explicitly decide *not* to do, and why? Where did you tighten the spec? Where did you loosen it?

The goal of the project was to design a backend application capable of matching lost items with found items using AI-assisted image understanding and embedding similarity. Users submit an image together with optional descriptive text. The system analyzes the image using a vision-language model (VLM), produces a structured description, converts that description into an embedding vector, and compares it against items in the opposite pool using cosine similarity. The result is a ranked list of likely matches.

The project exposes two user-facing interfaces. The first interface is a **FastAPI HTTP API** with the following endpoints:

- `POST /items/lost`
- `POST /items/found`
- `GET /items/{id}/matches?k=N`
- `GET /items?status=...`
- `GET /health`

The second interface is a CLI that mirrors the same functionality:

- `register-lost`
- `register-found`
- `search-matches`
- `list`

For online execution we used **OpenAI** as both the VLM provider and the embedding provider. The system can also operate in offline mode using deterministic fake providers for grading-safe execution and automated testing.

We intentionally focused on backend functionality rather than frontend development. The assignment did not require a graphical user interface, so our effort concentrated on reliability, persistence, concurrency, testing, and reproducibility. We also tightened the specification by adding explicit validation rules, Docker deployment support, benchmark tooling, and offline deterministic tests.

1.2 Provider choice and the AI module contract

Required. Name the LLM, embedding, and any auxiliary provider you used, and the model identifiers (claude-sonnet-4-6, text-embedding-3-small, etc.). State what ai/ functions you call. *Confirm in one sentence that you did not modify the public interface of the provided ai/ package.*

Push deeper: how does your code defend against a malformed model response? What happens when the provider returns valid JSON that is semantically wrong (wrong topic enum, negative grams, etc.)?

We used the following provider configuration during final testing:

- LLM / VLM provider: OpenAI
- LLM model: gpt-4o-mini
- Embedding provider: OpenAI
- Embedding model: text-embedding-3-small

The application calls the provided AI package through wrapper methods implemented in `src/services/ai_service.py`. The main AI calls include:

- `ai.describe_item(...)`
- `ai.embed(...)`
- `ai.top_k(...)`
- `ai.cosine(...)`

We did not modify the public interface of the provided `ai/` package.

The service layer validates both inputs and outputs around AI calls. Uploaded files are checked before the provider is called, and the returned model response is validated before being passed deeper into the application. The wrapper also handles malformed provider responses, timeout failures, retry behavior, and transient HTTP errors.

Even when a provider returns syntactically valid JSON, the response may still be semantically incorrect. For example, the model may identify an object incorrectly or produce unexpected description fields. To reduce the impact of such issues, the system validates the structure of responses through typed models and isolates AI failures from the rest of the application through explicit exception handling.

2. Architecture

2.1 Module map

Required. An architecture diagram (boxes-and-arrows or sequence diagram) showing the main modules and their dependencies. Identify on the diagram: the boundary of the provided `ai/` package, your service layer that wraps it, your core business logic, your concurrency layer, your storage layer, and your CLI/HTTP entry points.

Interpret your own diagram: which arrows cross the AI-module boundary? Which arrows cross the storage boundary? If you had to swap providers (Anthropic → OpenAI), how many files would change?

Module-by-module summary (1–2 sentences each):

The architecture is organized into clearly separated layers:

- API / CLI
- ↓
- Core Matching Service
- ↓
- AI Service Wrapper
- ↓
- Provided AI Package
- ↓
- Storage Repository
- ↓
- PostgreSQL + Filesystem Storage

The main modules are:

- `src/api.py` — FastAPI HTTP API
- `src/cli.py` — command-line interface
- `src/core/matching_service.py` — main business logic
- `src/services/ai_service.py` — wrapper around provided AI module
- `src/storage/repository.py` — persistence layer
- `src/concurrency/pipeline.py` — bounded async concurrency
- `src/core/validation.py` — input validation
- `src/config.py` — typed configuration management

The boundary of the provided AI module is explicitly isolated. Only the AI service wrapper communicates directly with the `ai/` package. Storage access is similarly isolated behind repository classes. Because provider selection is configuration-driven, switching from OpenAI to Gemini would only require changing `.env` values rather than rewriting the main application logic.

- **API layer:** receives HTTP requests and returns structured JSON responses.
- **CLI layer:** mirrors API functionality for terminal-based usage. This allows the application to be operated without running the FastAPI server while still using the same backend services and workflows
- **Matching service:** coordinates registration, validation, AI calls, storage, and matching.
- **AI service:** wraps provider calls with retries, caching, logging, and concurrency control.
- **Repository layer:** persists metadata and retrieves stored items. It abstracts database access from the rest of the application and supports PostgreSQL, JSON fallback storage, and in-memory repositories for testing.
- **Concurrency layer:** manages bounded parallel processing. It uses `asyncio.gather` and `asyncio.Semaphore` to improve throughput while preventing excessive simultaneous provider requests.
- **Validation layer:** validates uploaded images and user input. It ensures that invalid images, oversized files, corrupted uploads, and malformed requests are rejected before entering the AI pipeline.
- **Configuration layer:** loads typed settings from environment variables.

2.2 OOP design & key abstractions

Required. Identify at least one example of inheritance or composition *in your own code* (not the provided package's). Quote 5–10 lines of the relevant class or object design. Justify why this abstraction was the right tool; if you used an ABC, explain why.

Defensibility: if a grader asked "why is this an abstract base class rather than a Protocol, a plain function with a callable parameter, or a configuration enum?" — what is your answer? Are there places where you used composition where inheritance would have been wrong, or vice versa?

Code listing:

```
1 • class MatchingService:
2 •     def __init__(
3 •         self,
4 •         repository,
5 •         ai_service,
6 •         settings,
7 •     ) -> None:
8 •         self._repo = repository
9 •         self._ai = ai_service
10 •         self._settings = settings
```

Justification (2–3 sentences):

This abstraction allows dependencies to be injected instead of created internally. As a result, the service becomes significantly easier to test because fake repositories and fake AI providers can be supplied during offline tests.

Composition was the correct choice because the matching service does not represent a “type of” repository or “type of” AI provider. Instead, it coordinates multiple independent components. Using inheritance in this situation would have tightly coupled unrelated responsibilities together and made testing harder.

We also reused the abstract provider design already present inside the provided AI package. The provider-agnostic structure allowed the application to switch between **OpenAI** and **Gemini** through configuration changes only.

2.3 Data flow across module boundaries

Required. List the dataclasses you defined for your own SE layer. Confirm in one sentence that no naked dictionaries cross module boundaries.

The hardest call: when did you reuse a dataclass from the provided ai/ package vs. wrap it in a new one for your SE layer? What was the trigger for wrapping?

The application defines its own typed data models in `src/models.py`. These models include:

- `ItemRecord`
- `ItemResponse`
- `MatchResponse`
- `ErrorResponse`

No naked dictionaries cross module boundaries inside the SE layer.

3. Concurrency & Performance

3.1 Why this concurrency model

Required. State which concurrency primitive you used (`asyncio`, `ProcessPoolExecutor`, `ThreadPoolExecutor`) and why. Identify the bottleneck the parallel design targets. State the bound (e.g. `Semaphore(10)`) and how you picked it.

Justify against alternatives: could you have used threads instead of `asyncio`? Why not? What is the smallest workload at which the concurrent design starts to win against sequential? What happens if you set the semaphore to 1 vs. 100?

We used `asyncio` together with `asyncio.gather` and `asyncio.Semaphore` because the project workload is primarily I/O-bound rather than CPU-bound. AI provider calls and database interactions spend most of their time waiting on network responses rather than consuming processor time.

The concurrency implementation is located in `src/concurrency/pipeline.py`. A **semaphore** is used to bound parallelism:

```
asyncio.Semaphore(concurrency_limit)
```

We selected bounded async concurrency because unrestricted parallelism could quickly exceed provider rate limits. During development, we experimented with larger concurrency values and observed increased provider instability and quota exhaustion.

Using threads would also have worked for some blocking SDK calls, but `asyncio` integrated more naturally with the FastAPI application and async storage layer. Sequential execution became noticeably slower once the workload exceeded only a few AI requests.

If the semaphore bound is set to 1, execution becomes effectively sequential. If the bound is set too high, provider rate limits and network overhead become the new bottlenecks.

3.2 Sequential-vs-concurrent benchmark

Required. A table reporting wall-clock time for the same workload run sequentially vs. concurrently. State N (number of requests), the machine, the python version, whether caches were cleared, and the command line. Reproduce the table also in your README.

Honest interpretation: did you reach the theoretical $N \times$ speedup? If not, what was the new bottleneck (provider rate limit? GIL? network RTT? single-threaded DB writes?). What would push the curve further?

Workload	N	Sequential (s)	Concurrent (s)	Speedup
e.g. register 20 items	20	4.00	0.8	5.00x
e.g. analyze 16 meals	16	3.20	0.64	5.00x

Discussion (2–3 sentences):

The benchmark demonstrates that bounded asynchronous concurrency significantly improves throughput for I/O-bound workloads. Although the theoretical speedup could be higher, network overhead, event-loop scheduling, and concurrency limits reduce the maximum achievable gain. The concurrent implementation consistently outperformed sequential execution once the workload involved several parallel tasks. In practice, provider rate limits and external API latency become the primary bottlenecks rather than CPU performance.

3.3 Rate limit & politeness

Required. State the rate-limit strategy (sleep-on-429, token bucket, leaky bucket, request budget). Document the configured budget. State the highest burst your test suite produces.

Failure-mode test: have you actually observed a 429 from the provider during development, or is this strategy untested in anger? What does the log look like when the budget is hit?

The system uses bounded concurrency together with retry-based backoff handling. Parallel AI calls are restricted using `asyncio.Semaphore`, which prevents the application from sending an excessive number of simultaneous requests.

Transient provider failures are retried using exponential backoff:

`backoff = 0.5 * (2 ** attempt)`

During development we encountered real provider quota and rate-limit failures, especially while testing Gemini free-tier limits. The logs clearly showed retry attempts, delays, and provider responses. This helped us verify that retry handling and concurrency bounding worked as intended.

4. Robustness & Error Handling

4.1 Wrapping the AI modul

Required. Describe your service-layer wrapper around ai.* calls: retries (which exceptions, how many attempts, backoff schedule), timeouts (per-call value), structured logging (what

Pressure-test: what happens if the provider returns a 500? A 429? A connection reset mid-stream? A perfectly-formed JSON that contradicts the schema? Walk through one of these end-to-end.

The AI wrapper in `src/services/ai_service.py` adds retries, timeout handling, logging, caching, and concurrency control around the provided AI package.

Features implemented by the wrapper include:

- retries with exponential backoff
- timeout protection
- structured logging
- embedding caching
- bounded concurrency
- provider-independent execution
- conversion of provider failures into application-level exceptions

Timeouts are enforced around provider calls. Transient failures such as provider 5xx responses or connection resets trigger retries. Validation occurs before provider calls and after responses are returned.

If a provider returns malformed or semantically incorrect data, the system raises application-level exceptions instead of allowing invalid values to silently propagate through the application.

4.2 Failure-mode analysis (one concrete incident)

Required. Pick *one* failure mode you actually exercised (provider 5xx, timeout, malformed response, network drop, disk full, corrupted image, USDA outage, etc.). Describe: (i) how you injected the failure, (ii) what the system did, (iii) what the user saw, (iv) what the logs showed.

Go beyond "graceful degradation worked": what did you originally expect to happen vs. what actually happened? Did the test surface any design issue that you then fixed? Did the log give you enough to diagnose it without running the code again?

One failure mode we intentionally exercised during development was provider quota exhaustion.

We triggered this failure by repeatedly calling the online demo pipeline using free-tier provider accounts. The provider responded with HTTP 429 rate-limit errors.

The system did not crash. Instead:

- retries were triggered automatically
- exponential backoff delays were applied
- errors were logged clearly
- the user received structured failure information

Originally, we expected the application to fail immediately. Instead, the retry logic exposed how important bounded concurrency and provider-specific behavior were for system stability. This led us to improve logging and benchmark tooling.

4.3 Input validation

Required. For every entry point (CLI, HTTP, file, env), list the validation rules. State what error message the user gets when validation fails.

Adversarial: could a malformed image kill your service? A 100 MB JSON request? A query of 50,000 characters? A path that escapes your storage directory?

[Validation rules are enforced for every entry point.

HTTP API validation:

- valid image upload required
- JPEG/PNG MIME type only
- maximum image size enforced
- corrupted images rejected
- invalid item IDs rejected

CLI validation:

- image path existence
- readable image file
- valid command arguments

Environment validation:

- typed settings through `pydantic-settings`
- missing required provider configuration detected at runtime

The system rejects malformed input with clean error messages instead of exposing stack traces.

Large invalid payloads, unsupported file formats, corrupted images, and invalid paths are all handled safely.

5 Storage & Caching

5.1 Schema

Required. The SQLite schema for your domain tables. Include indices and foreign keys. State which fields are derived (cached) vs. source-of-truth.

Schema choices: why did you store embedding bytes as a blob rather than serialized JSON, or vice versa? What happens to the DB if the embedding dimension changes (a different provider, a different model)?

```
CREATE TABLE items (  
    id TEXT PRIMARY KEY,  
    status TEXT NOT NULL,  
    user_text TEXT NOT NULL,  
    image_path TEXT NOT NULL,  
    vlm_description TEXT NOT NULL,  
    embedding BLOB NOT NULL,  
    created_at TEXT NOT NULL  
);
```

5.2 Caching policy

Required. What do you cache, how long, where, and how is the cache invalidated. Report cache hit rate from a demo run (a single number is fine).

Correctness vs. freshness: which cached values can become stale (e.g. USDA nutrition for a renamed food, an LLM summary if the prompt changes)? How would you detect and evict them in production?

The application caches embedding vectors inside the AI service during runtime.

Cached values:

- embedding vectors for repeated descriptions

Cache location:

- in-memory session cache

Cache invalidation:

- cache cleared when application restarts

The cache significantly reduces repeated provider calls when identical descriptions are processed multiple times.

Cached values may become stale if providers, prompts, or embedding models change. In a production system, embedding versioning or provider identifiers would likely be stored together with cached vectors.

6 Testing

6.1 Strategy & coverage

Required. Report the coverage number (pytest -cov). State how you mocked the AI module and the HTTP layer. Confirm the provided tests/test_ai_smoke.py still passes.

Quality vs. coverage: which lines did you decide *not* to cover, and why? Which mocks did you have to redesign mid-project because they were testing the wrong thing?

Suite	Coverage
src/ (your SE layer)	76%
Provided ai/ smoke tests	<i>passing? yes</i>

6.2 Notable tests

Required. Highlight at least three tests: (i) one happy-path end-to-end test, (ii) one concurrency test (e.g. that `asyncio.gather` behaves correctly when one task raises), (iii) one error-path test that surfaces a real bug you would otherwise have missed.

What didn't you test? What is the test you wish you had time to write?

One important **end-to-end test** verifies that a lost item and a found item can be registered through the FastAPI application and successfully matched.

A **concurrency test** verifies that the batch pipeline correctly handles multiple async tasks and preserves behavior even when one task fails.

An **error-path test** verifies that invalid image uploads return validation errors instead of crashing the API.

One test we would have liked to add is a heavier load test simulating large numbers of simultaneous registrations against a real provider.

7 Deployment & Reproducibility

7.1 Docker image

Required. The exact docker build and docker run commands. The image size. The Python version baked in. Note whether you use a single-stage or multi-stage build.

Engineering trade-offs: did you ship an Alpine, slim, or full base image, and why? How does your image size compare to a fresh `python:3.12-slim` (~120 MB)?

The application is containerized using **Docker**.

Build command:

```
docker build --no-cache -t lost-found-ai .
```

Run command:

```
docker run --env-file .env.docker -p 8000:8000 lost-found-ai
```

The project uses a single-stage Docker build based on:

```
python:3.12-slim
```

We selected the slim image because it provides a good balance between image size and compatibility.

7.2 Configuration

Required. A short list of every environment variable the app reads, and what happens if each is missing.

The application reads the following environment variables:

- LLM_PROVIDER
- LLM_MODEL
- OPENAI_API_KEY
- GOOGLE_API_KEY
- ANTHROPIC_API_KEY
- EMBEDDING_PROVIDER
- EMBEDDING_MODEL
- DATABASE_URL
- IMAGE_STORAGE_DIR
- MAX_IMAGE_SIZE_MB
- HTTP_PORT
- LOG_LEVEL

If provider configuration is missing, online AI execution fails with a clear provider error. If PostgreSQL is unavailable during local development, the application can fall back to JSON storage.

8 Limitations & Future Work

Required. 3–5 bullet points. Honest list of where the system is brittle, what it would take to productionize, and what you would do differently with another week.

Be honest: no project is perfect. What was the worst design decision you made? What did you not get to test under load? What is the scenario most likely to break the system on a busy day?

- The matching quality still depends heavily on provider-generated descriptions.
- Real provider quotas and rate limits can affect online execution.
- The system has not been tested under very large production-scale workloads.
- The current embedding cache exists only in memory and resets on restart.
- A frontend interface and authentication system were intentionally left out of scope.

If we had another week, we would focus on better observability, load testing, and persistent distributed caching.

9 Tools & Acknowledgements

Required. Disclose substantial AI-assistant use. For each significant LLM-aided contribution: which module, which assistant (Claude/ChatGPT/Cursor/Copilot/etc.), and whether the team reviewed and adapted it.

Cultural note: "AI helped with everything" is not a useful disclosure; "Cursor drafted the initial `retry_policy.py`; we rewrote backoff logic after observing it hit the rate limit" is.

AI assistance was used throughout the project for debugging, documentation drafting, benchmarking support, and implementation guidance.

Tools used included:

- ChatGPT
- Cursor
- OpenAI API
- Docker
- PostgreSQL
- FastAPI
- pytest

AI-generated suggestions were reviewed, adapted, tested

References

- [1] <https://developers.openai.com/api/docs/guides/embeddings>
- [2] <https://docker-docs.uclv.cu/>
- [3] <https://devdocs.io/fastapi/>
- [4] https://pydantic.dev/docs/validation/latest/concepts/pydantic_settings/
- [5] <https://medium-api.readthedocs.io/en/latest/>
- [6] <https://developers.openai.com/api/docs/models/text-embedding-3-small>

A. Appendix — Reproducing the benchmark

Exact commands to reproduce the ğ3 benchmark. Anyone with the repo and a valid .env should be able to run the lines below and get numbers within 20% of the ones in the table.

```
$ git clone https://github.com/muradtto11/item_lost_found_ai_project.git
$ cd item_lost_found_ai_project
$ cp .env.example .env
# Fill in OPENAI_API_KEY if online AI execution is needed.
$ python3 -m pip install -r requirements.txt
$ python3 scripts/benchmark.py
```

End of report — thank you for reading.